Tempo-Relational Neural Networks for team modeling in Operating Room

Vincenzo Marco De Luca, University of Trento

Bulletpoints

1) Human-Machine Teaming
2) Modeling teams
3) Explainability
4) Preliminary results

Human-Machine Teaming

AI-Coach in Human-Machine Teaming (HMT)

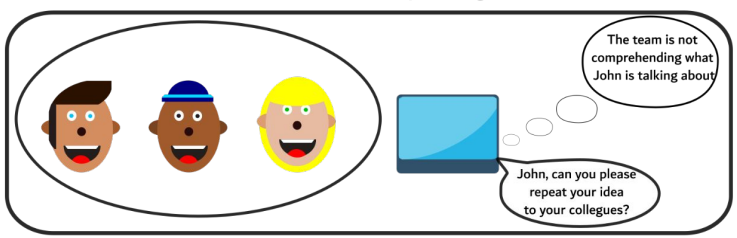


Figure: AI-Coach in HMT based on the principles of Theory of Mind AI and Human-Centered AI

AI-Coach capabilities:

+ Automate **Team Analysis**
+ Predict **Team Construct**
+ Off-line Explanation of Team Performance
+ Online Suggestion to enhance performance

Human-Machine Teaming

AI Coach for High-Stake HMT

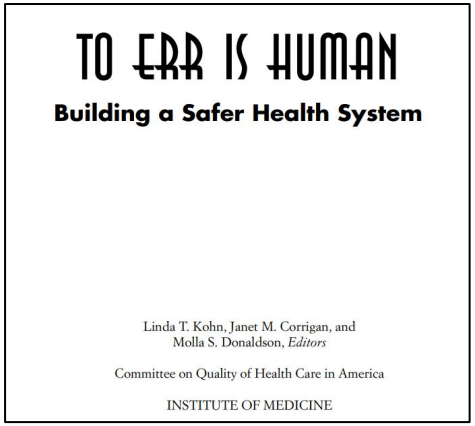
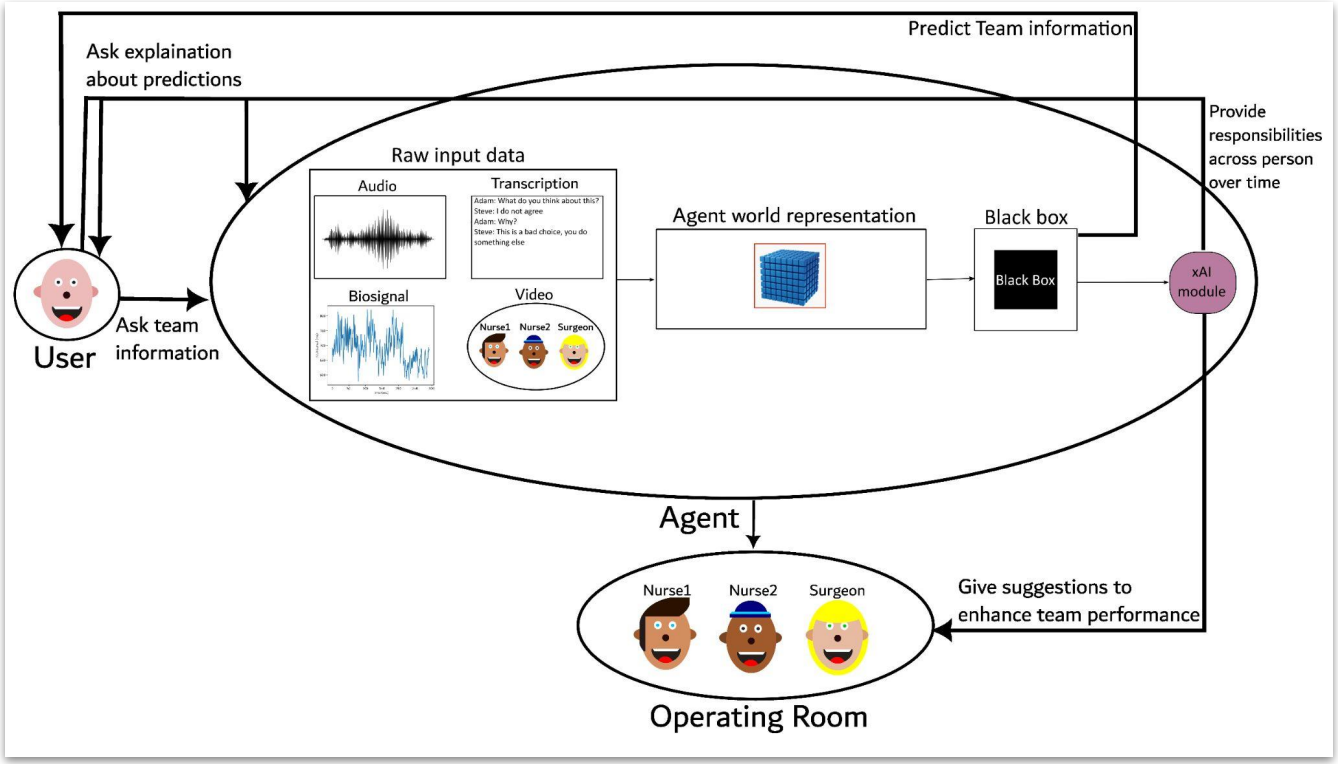


Figure: (Brown, 2001)

- Medical errors are the **third leading cause of death** in the US (Makary, 2016)
- One of the main reason for medical errors is **Teamwork in the Operating Room** (Pronovost 2013)
- Design **agent able to enhance Teamwork, Team efficiency, and Team-Decision Making**

4

Human-Machine Teaming



Human-Machine Teaming

Rating schema

Given a multimodal dataset (audio and video) for each team member we measure:

1) Teamwork: Adaptability, Back-Up Behavior, Communication, Mutual Trust, Performance Monitoring, Shared Mental Model, Team Orientation,

2) Leadership Style: Dominance, Friendliness, Task Orientation

3) Emergent Leadership

Annotation frequency: Fill each rating schema every three minutes

Modeling teams

Computational Models

Static Neural Networks

Extract features to represent each team member, and predict team-member information.

Temporal Neural Networks

Extract features for each team member and learns weight to integrate dynamics

Relational Neural Networks

Extract features for each team member and connect team members based on their interactions

Social Science theory

Relational aspect must be integrated in modeling teams

Temporal aspect is crucial for the team dynamics and team processes

There are **shared features** across multiple team processes and behaviors

Modeling teams

Encoder-Decoder perspective

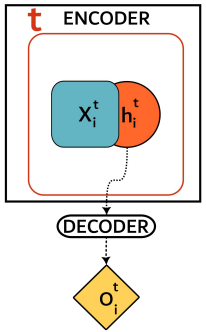
+ The encoder: aims at extracting **Social Embedding.**

  Each team member must be mapped into a feature vector that accounts for interactions and dynamics

+ The decoder: aims at predicting social information such as team processes and teamwork

Modeling teams

Encoder - Static Neural Networks



Definition 6. A *static team member* is defined as a pair $\{\mathbf{x}, \mathbf{y}\}$ where $\mathbf{x} \in R^n$ are the input speech features associated with the team member, whereas $\mathbf{y} \in R^m$ are the output information described as a score (e.g. emergent leadership, leadership style, teamwork components).

This data composes the datasets used to train *SNNs*.

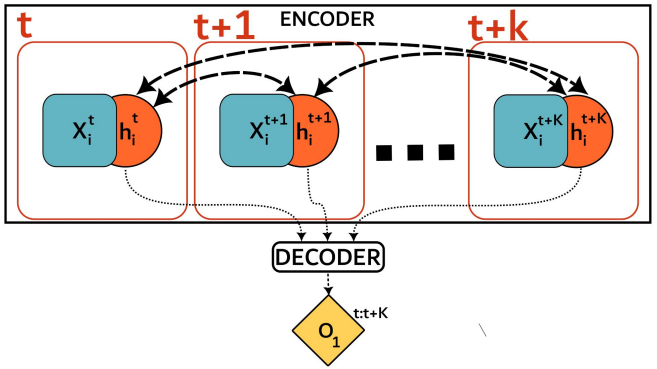
Definition 7. A *Static Neural Network* (SNN) is defined as a set of parameters $\{W, \mathbf{b}\}$, that maps a *static team member* into its corresponding outcome.

$$\mathbf{h} = f_{\text{snn}}(\mathbf{x}) = W \cdot \mathbf{x} + \mathbf{b} \quad (1)$$

LEGEND input hidden representation output edge temporal connection forward hidden representation to MLP time-step

Modeling teams

Encoder - Temporal Neural Networks



Definition 8. A *dynamic team member* is defined as pair $\{X, \mathbf{y}\}$, where $X = (\mathbf{x}^1, \mathbf{x}^2, ..., \mathbf{x}^t)$ where a sequence of speech features for a specific team member over t consecutive time steps $\mathbf{x}^j \in \mathscr{R}^n$ is associated with a single outcome associated over the entire sequence $\mathbf{y} \in \mathscr{R}^m$.

TNNs are trained on dataset composed of *dynamic team members* because they encode temporal dynamics This type of data composes datasets used to train *TNNs*:

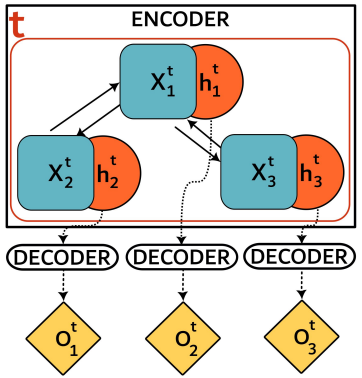
Definition 9. A *Temporal Neural Networks* (TNNs) is defined as a set of parameters used to learn temporal dependencies over time $\{\mathbf{A}\}$, encoding these temporal dependencies, into the resulting embedding given a *dynamic team member*.

$$\mathbf{h} = f_{\text{tnn}}(\mathbf{A}, (\mathbf{x}^1, \mathbf{x}^2, ..., \mathbf{x}^t)) \quad (2)$$

LEGEND input hidden representation output edge temporal connection forward hidden representation to MLP time-step

Modeling teams

Encoder - Relational Neural Networks



Definition 10. A *static team* is defined as a tuple, representing a graph: $G = (\mathscr{V}, \mathscr{E}, \mathbf{X}, \mathbf{y})\}$, where the set of team members in the team is $\mathscr{V} = \{v_1, v_2, ..., v_n\}$ and n denotes the number of team members. The set of interactions between team members is $\mathscr{E} \subseteq V \times V$. The set of speech features is $\mathbf{X} = \{\mathbf{X}_v | v \in V\}$, where $X_v \in \mathscr{R}^n$, and $\mathbf{y} = \{\mathbf{y}_v | v \in V\}$ is the output associated to each team member v at a specific time instant, where $y_v \in R^m$.

This type of data composes the datasets used to train *ReNNs*:

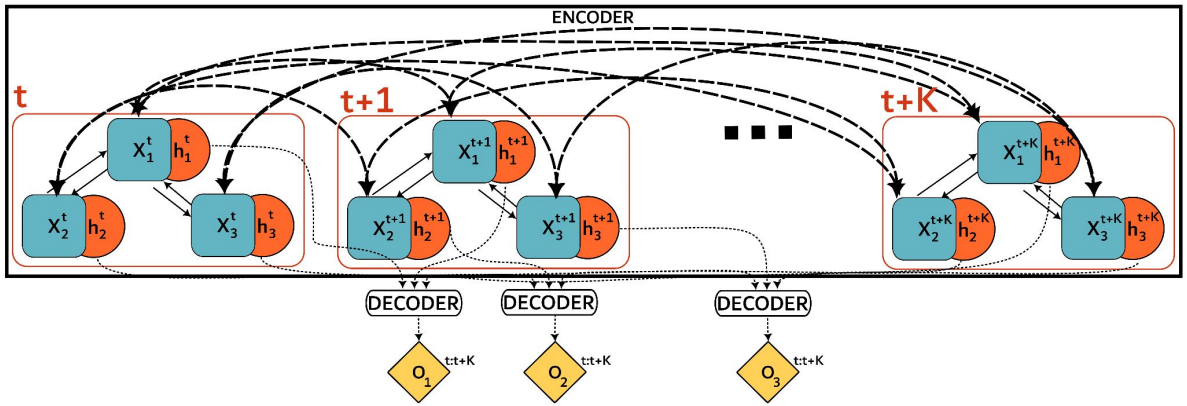
Definition 11. A *Relational Neural Networks* (ReNNs) is defined as a set of parameters used to learn relational dependencies among team members ($\mathbf{W}_r$, $\mathbf{b}_r$), given a *static team* $G = (\mathscr{V}, \mathscr{E}, \mathbf{X})$ as input, to encode each team member in the team into its corresponding embedding $\mathbf{h}_v$

$$\mathbf{h}_v = f_{\text{renn}}(\mathbf{W}_r, \mathbf{X}_u \mid u \in \{v \cup \mathscr{N}(v)\}) \ , \ \forall v \in \mathscr{V} \qquad (3)$$

LEGEND input hidden representation output edge temporal connection forward hidden representation to MLP time-step

Modeling teams

Encoder - Tempo-Relational Neural Networks



Definition 12. A *dynamic team* is defined as a pair $T = (\mathbf{G}, \mathbf{Y})$, where $\mathbf{G} = \{G^1, G^2, ..., G^t\}$ is the sequence of team interaction, and each of them is defined as a triplet $G^j = (\mathcal{V}^j, \mathcal{E}^j, \mathbf{X}^j)$, where $\mathcal{V}^j$ is the set of team members at time j, $\mathcal{E}^j \subseteq \mathcal{V}^j \times \mathcal{V}^j$ is the set of interactions between team members, and the individual team member information $\mathbf{X}^j = \{X_v^j | v \in \mathcal{V}^j\}$, and $\mathbf{Y} = \{y_v | v \in \mathcal{V}^j\}$.

This data structure captures a *static team* in time, in such a way that its team member properties and interaction may evolve over consecutive time steps. This type of data composes the dataset used to train TReNNs as reported in Fig. 3(d).

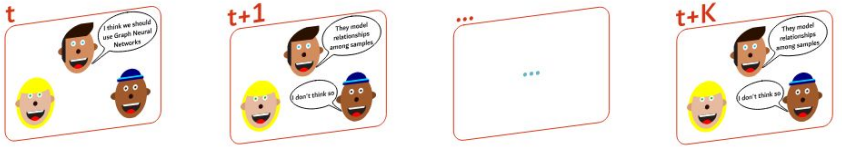
Definition 13. A *Tempo-Relational Neural Network* (TReNNs is defined as a set of parameters used to learn relational dependencies across team members $(\mathbf{W}_r, \mathbf{b}_r)$, the parameters used to learn temporal dependencies of a team member over time $\{\mathbf{A}\}$.

$$h_v^{1:t} = f_{\text{trenn}}\left((A, W_r), (G^1, G^2, ..., G^t)\right) \quad (4)$$

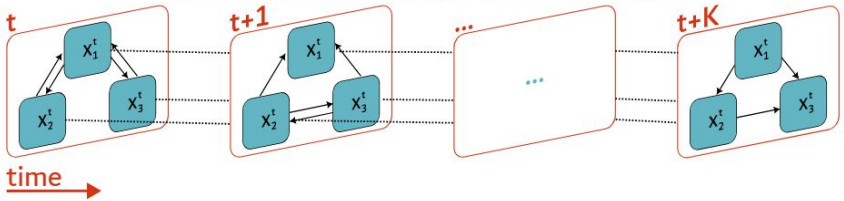
LEGEND input hidden representation output edge temporal connection forward hidden representation to MLP time-step

Modeling teams

Step 1 - Build the Temporal Graph



((a)) Real world Interaction: How team interact in a continuous and analogic real-world



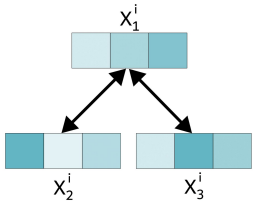
((b)) Snapshot Temporal Graph: How the agent sees the real-world under a discrete and digital perspective

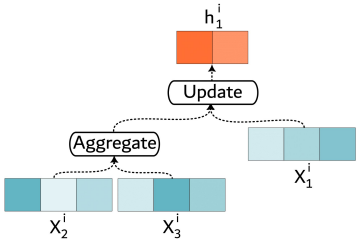
+ Extract features from non-verbal behavior (single frame or speech) and verbal behavior (text) in a fixed time-window (e.g. 15 sec)

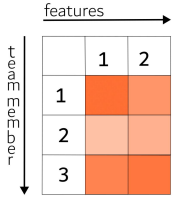
+ Connect pair of team-member with weighted edges based on their non-verbal communication (e.g. gaze, relative distance, speech) or verbal communication (e.g. whom is addressed a message to)

Modeling teams

Step 2 - Learning from Relations







The *Spatial GNN (MPNN)* project the features of each node (representing a team member) into the embedding space
Message Passing is realized across two steps:

1) Message

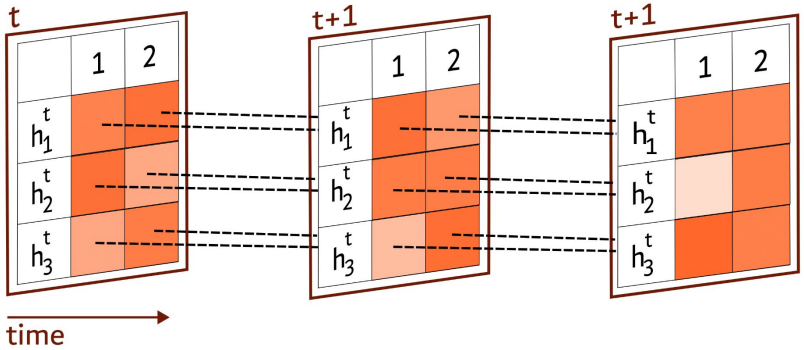
$$m_v^{(l)} = \sum_{u \in N(v)} M^{(l)} \left(h_v^{(l-1)}, h_u^{(l-1)}, e_{vu} \right)$$

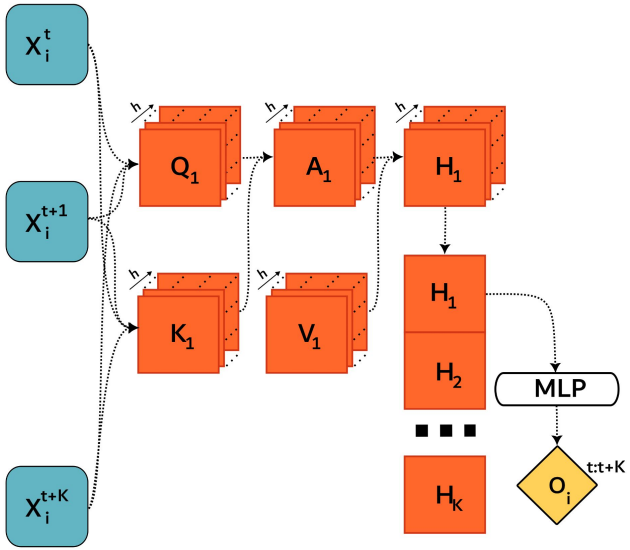
2) Update

$$h_v^{(l+1)} = U^{(l)} \left(m_v^{(l)}, h_v^{(l)} \right)$$

Modeling teams

Step 3 - Learning over time





+ Based on the Multi-Head Attention paradigm, given the node embedding, the TGNN produces for each node an embedding that accounts for the team dynamics

$$A = \text{Attention}(Q, K, V)_h = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \ , \forall h \in [1, H]$$

Modeling teams

Encoder - Contrastive Learning

Often qualitative annotation is noisy, then rather than accounting only for absolute scores on individual subjects, we may introduce the ranking of the team members according to their corresponding scores.

The underlying idea is that:

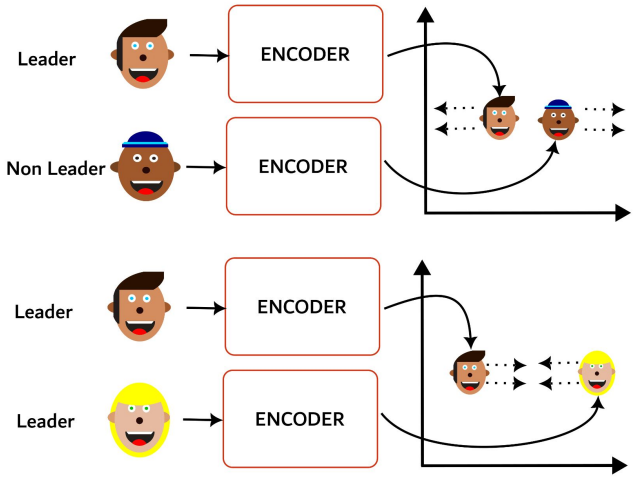
+ the most leaders in two different teams should be closer rather in the embedding space
+ the most leader and the least leader in two different teams should be farther in the embedding space

This approach gives more importance to the ranking of the team members, as it would be a classification task, rather than their absolute scores.

Moreover enables to leverage an additional information from the labeling procedure

Modeling teams

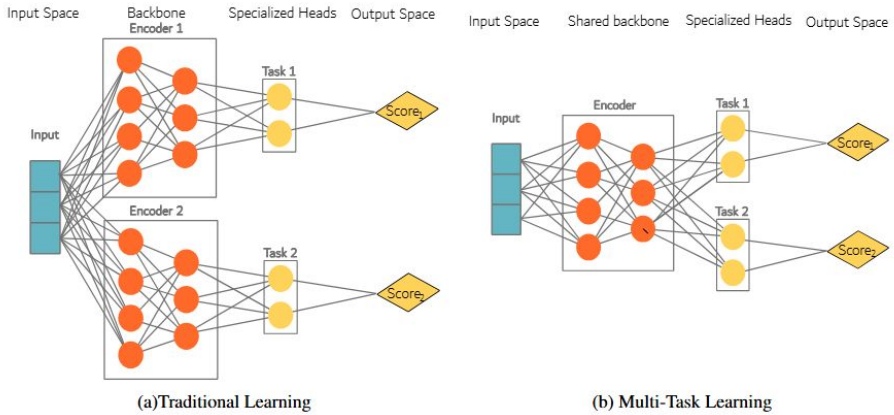
Encoder - Contrastive Learning - Pairwise Loss



$$L_p = \sum_{(i,j)\in D} \max(0, 1 - (f(\mathbf{x}_i) - f(\mathbf{x}_j)))$$

Modeling teams

Decoder - Multi-Task Learning



(a)Traditional Learning

(b) Multi-Task Learning

+ Given the node embedding for each team member (produced by the shared backbone, Encoder), each specialized head (Decoder) learns different parameters to map the node embedding into the final score for each of the team process.

+ The optimization procedure is defined as the weighted contribution of each single task loss:

$$L = \sum_{j=1}^{T} \gamma_i \cdot l_i$$

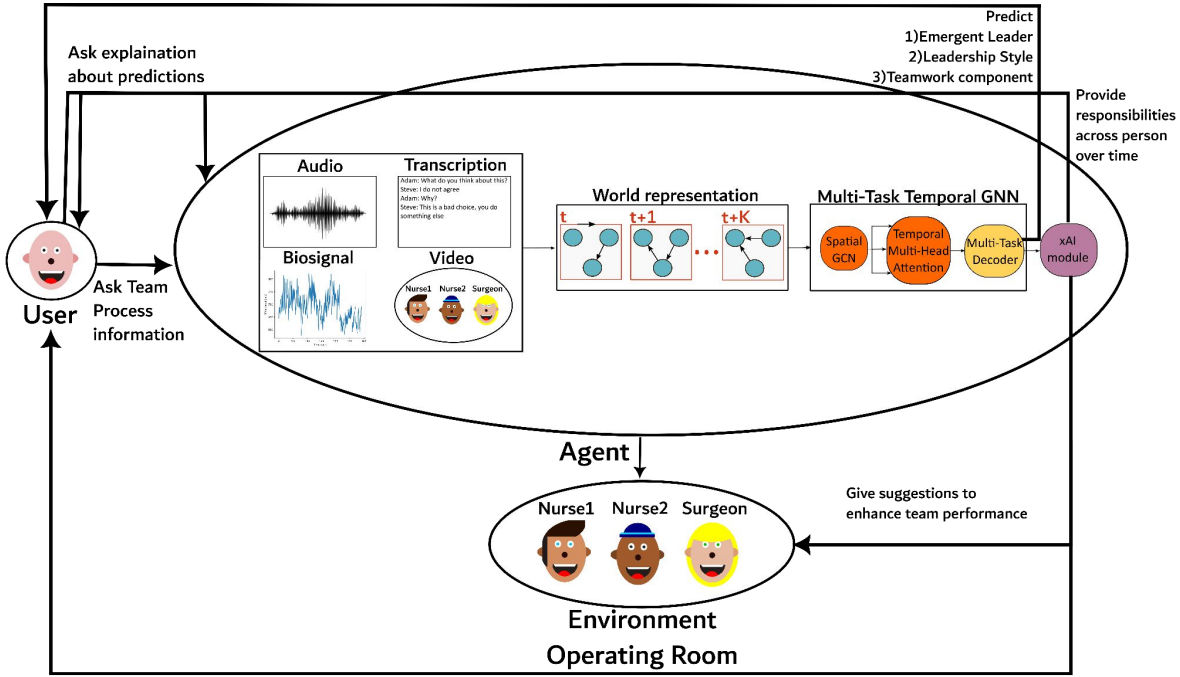
Modeling teams

Final overview - Encoder + Decoder

Architecture	Relational	Temporal	MultiTask
SNN	✗	✗	✗
TNN	✗	✓	✗
ReNN	✓	✗	✗
TReNN	✓	✓	✗
MT-TReNN	✓	✓	✓

Modeling teams

Final overview



Features and topology

Graphs are extracted every 15 seconds:
Each node is a person

Speech/text-based features:

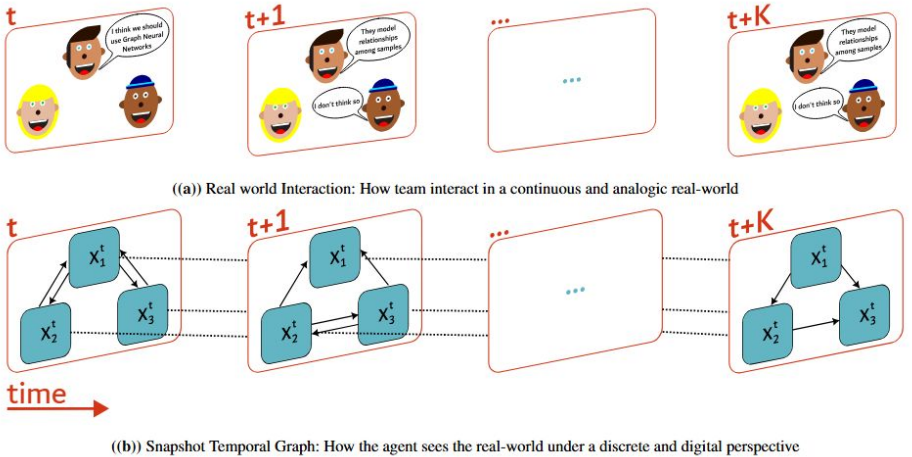
1) EGeMaps features from speech
2) Text embedding

Speech/text-based edges:

1) Overlap
2) Turn taking
3) Speaker Broadcast

CV-based edges:

1) Gaze
2) Relative distance



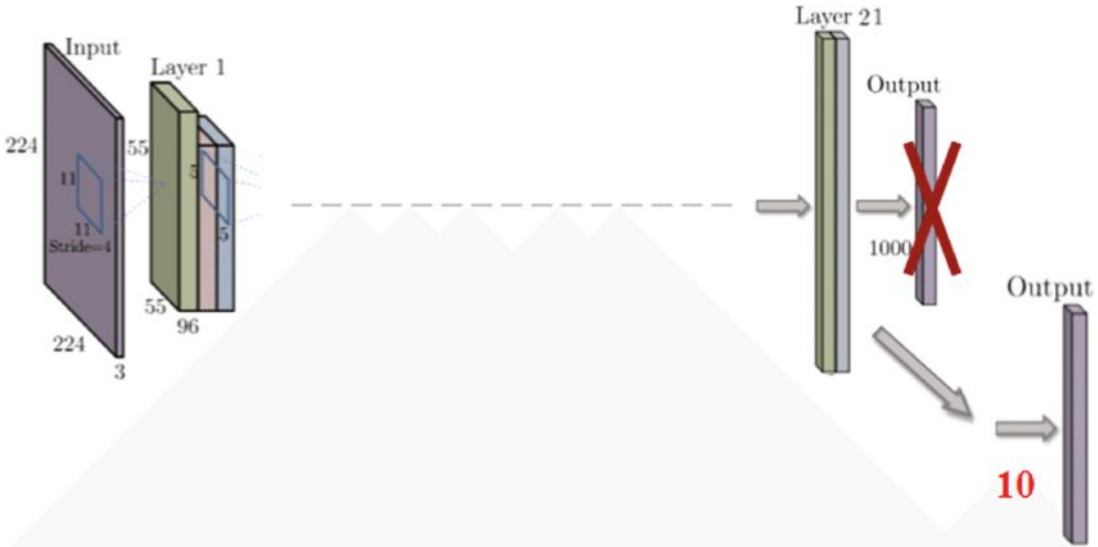
((a)) Real world Interaction: How team interact in a continuous and analogic real-world

((b)) Snapshot Temporal Graph: How the agent sees the real-world under a discrete and digital perspective

Modeling teams

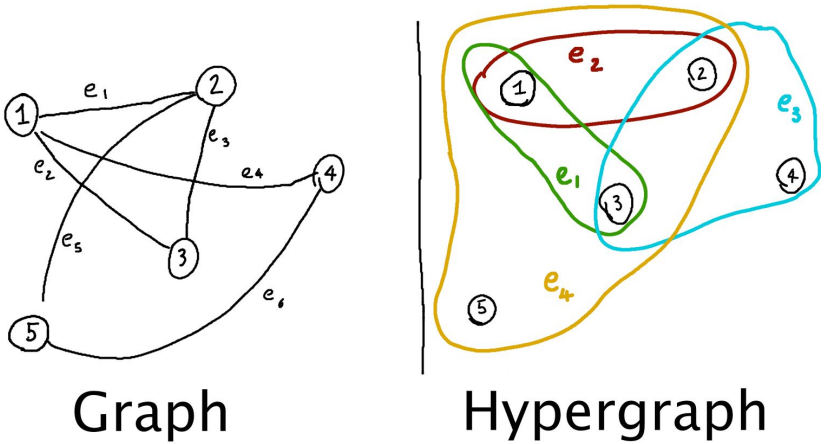
Case 1 - Finetuning

Does Social Embedding generated from general purpose domain generalize to Operating Room (OR) domain? Despite we are dealing with out-of-distribution graphs?



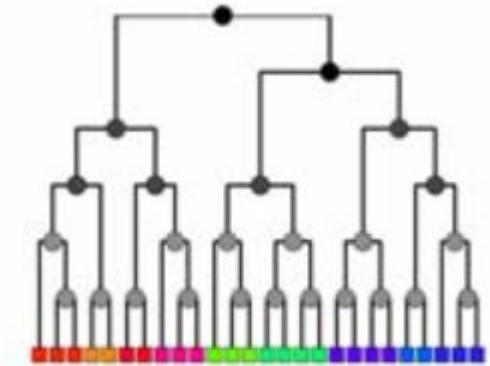
Modeling teams

Case 2 - From Graphs to Hypergraphs



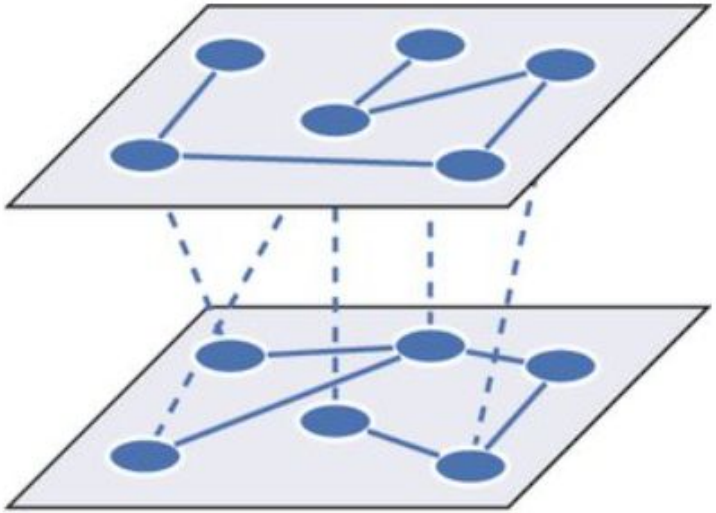
Graph topology and modeling

Case 3 - Hierarchical Graph



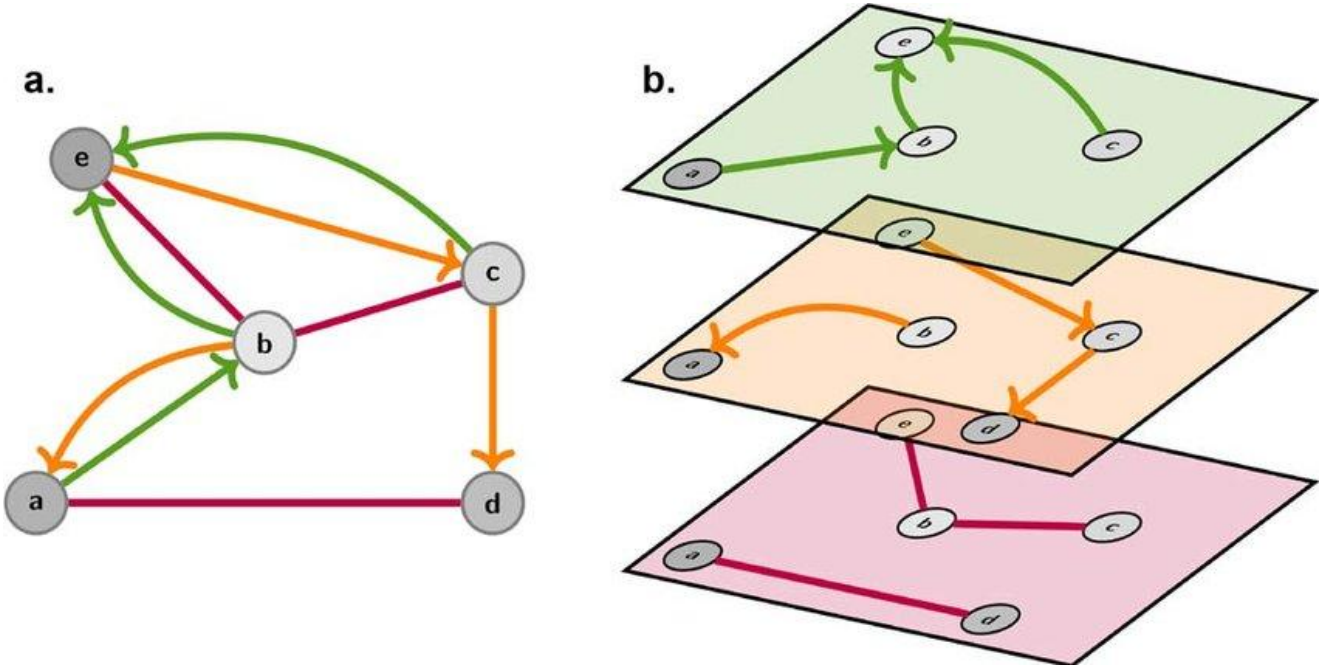
Modeling teams

Case 4 - Multi-Layer Graphs for Multimodality



Modeling teams

Case 5 - Multi-Relational Graphs for Multimodality



Explainability

LLM for providing intuitive explanations

GraphXAIN: Narratives to Explain Graph Neural Networks

Mateusz Cedro*[1] and David Martens[1]

[1]University of Antwerp, Belgium

Explainability

Why do we need explanations?

1) Social Scientists need explanation for proving their theories
2) Organizational psychologists need explanations for enhancing team quality
3) Domain experts (e.g. surgeons) need explanations for comprehending team phenomena

Which types of explanations?

0) Factual: Tell why do we get a certain prediction
1) Counterfactual: How to get another prediction rather than the current one

Explanation granularity

An explanation may be either local (single prediction level) or global (tell what the model has learned for a class).

Explainability

Produce (local) explanations

Saliency Map calculates the magnitude of the gradients with respect to node features. Large gradients indicate features that significantly impact the model's output when perturbed. Formally, it is defined as:

$$G_{exp}(v) = \frac{\partial f_v(G, v)}{\partial X_v}$$

where Xv represents the feature vector corresponding to the node $v \in G$. In a TGNN, where the input evolves across multiple time steps, this approach also reveals which time steps are most influential.

We have considered this GNN explainer because it is one of the most efficient and can produce real-time explanations

Explainability

Temporal Graph explanations

a) Feature-level explanations to tell which feature affect team phenomena
b) Node-level explanations to tell who affect team phenomena
c) Edge-level explanations to tell which interactions affect team phenomena
d) Subgraph-level explanations to explain some team phenomena at higher-level
e) Time-level explanations to tell which stages affect the most team phenomena

Some results

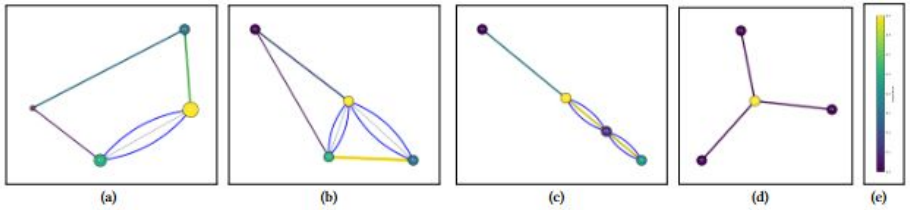
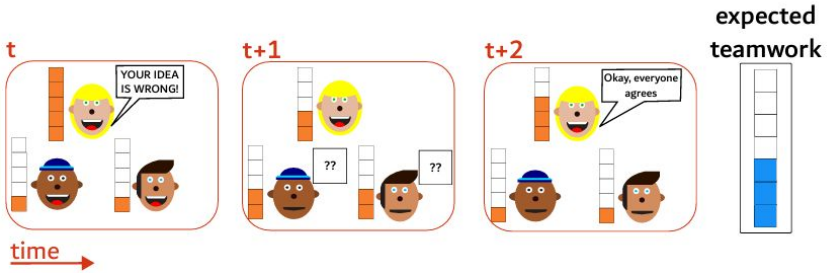


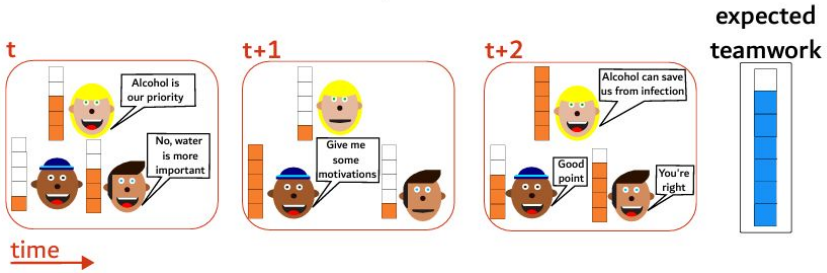
Fig. 2. Soft mask explanation produced by Saliency Map on Teamwork dimensions, more specifically in the example do we report the Soft mask explanation produced by exploring Adaptability prediction over time for a given team. (a)shows the explanation produced at time 0, (b) the explanation mask produced at time 1, (c) the explanation mask produced at time 2, (d) the explanation mask produced at time 3, (e) describes how the colors encode importance values in the explanation masks

Explainability

Factual explainability



(a) Example of Bad Teamwork



(b) Example of Good Teamwork

Fig. 12: How Team Interaction may affect teamwork

Explainability

Counterfactual explainability

Algorithm 1: GreeDy search algorithm for counterfactual examples.

Input: Link prediction function f, input graph $\mathcal{G}$, explained event ε_i, selection strategy δ, number of nodes to sample in each iteration l

Output: best explanation found $\mathcal{X}$

$prediction_{orig} \leftarrow f(\mathcal{G}(t_i), \varepsilon_i)$;
$n_{root} \leftarrow (\varnothing, prediction_{orig}, null, \varnothing)$;
$n_{best} \leftarrow n_{root}$;
while s_{best} does not include all candidate events $C(\mathcal{G}, \varepsilon_i, k, m_{max})$ **do**
  $children_{best} \leftarrow$ set of l child node with highest rank according to δ, each child n_{child} initialized with prediction $prediction_{child} = f((\mathcal{G}(t_i) \setminus s_{child}), \varepsilon_i)$;
  $n_{best_child} \leftarrow \arg\max_{n_j \in children_{best}} \Delta(prediction_{orig}, prediction_j)$;
  if $\Delta(prediction_{best}, prediction_{best_child}) > 0$ **then**
    /* n_{best_child} shifts the prediction further towards the opposite sign of the original prediction */
    $n_{best} \leftarrow n_{best_child}$;
    if $\Delta(prediction_{orig}, prediction_{best}) > |prediction_{orig}|$ **then**
      /* s_{best} is a counterfactual example */
      break;
    end
  else
    break;
  end
end
return s_{best}

Explainability

Counterfactual explainability

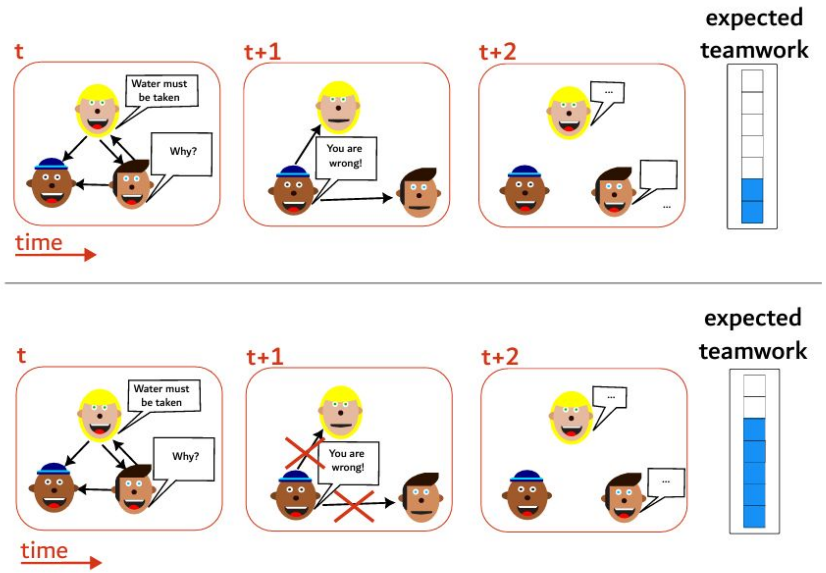


Fig. 13: Counterfactual example: In the top panel of the figure, a real world scenario is reported. In the bottom panel of the figure, the suggestion generated as a counterfactual explanation by CoDy is reported.

Preliminary results

Tempo-Relational Neural Networks - Emergent Leadership

Paradigm	Model	ACC@1 - Emergent Leadership PAVIS	ACC@1 - Emergent Leadership ELEA	ACC@Last - Emergent Leadership PAVIS	ACC@Last - Emergent Leadership ELEA
SNN	RF	79.54±0.18	70.28±0.12	78.97±0.53	71.28±0.47
	FFNN	77.05±0.54	67.68±0.94	78.24±0.72	70.11±0.65
TNN	LSTM	80.69±0.72	71.39±0.72	82.06±0.68	72.29±0.58
	MHA	81.25±0.56	72.48±0.52	83.37±0.45	74.98±0.44
RENN	GCN	83.96±0.64	73.18±0.76	83.96±0.64	74.22±0.48
TRENN	GCN+MHA	**85.95**±0.48	**74.55**±0.48	**85.95**±0.58	**75.42**±0.40

Table 5: The average ACC@1 and ACC@Last of Emergent Leadership over 10 different seeds. The significantly best-performing model is boldfaced. Different models are grouped according to their main modeling class (i.e., SNN, TNN, RENN, TRENN)

Preliminary results

Tempo-Relational Neural Networks - Leadership Style

Paradigm	Model	PAVIS			ELEA		
		Dominance	Friendliness	Task orient.	Dominance	Friendliness	Task orient.
SNN	RF	0.661±0.008	0.605±0.009	1.023±0.007	0.729±0.010	0.738±0.009	0.958±0.008
	FFNN	0.738±0.009	0.652±0.011	1.245±0.011	0.758±0.013	0.782±0.0010	0.933±0.015
TNN	LSTM	0.589±0.006	0.592±0.005	0.767±0.007	0.685±0.009	0.715±0.011	0.891±0.009
	MHA	0.538±0.005	0.548±0.008	0.738±0.005	0.678±0.004	0.643±0.009	0.843±0.008
RENN	GCN	0.513±0.004	0.503±0.005	0.701±0.004	0.638±0.008	0.652±0.010	0.839±0.009
TRENN	GCN+MHA	**0.488**±0.005	**0.485**±0.006	**0.675**±0.007	**0.575**±0.005	**0.582**±0.006	**0.788**±0.008

Table 6: Average MSE of Leadership style over 10 different random seeds. The significanlty best-performing method is boldfaced. Different models are grouped according to their main modeling class (i.e., SNN, TNN, RENN, TRENN)

Preliminary results

Social Embedding and Multi-Task Learning

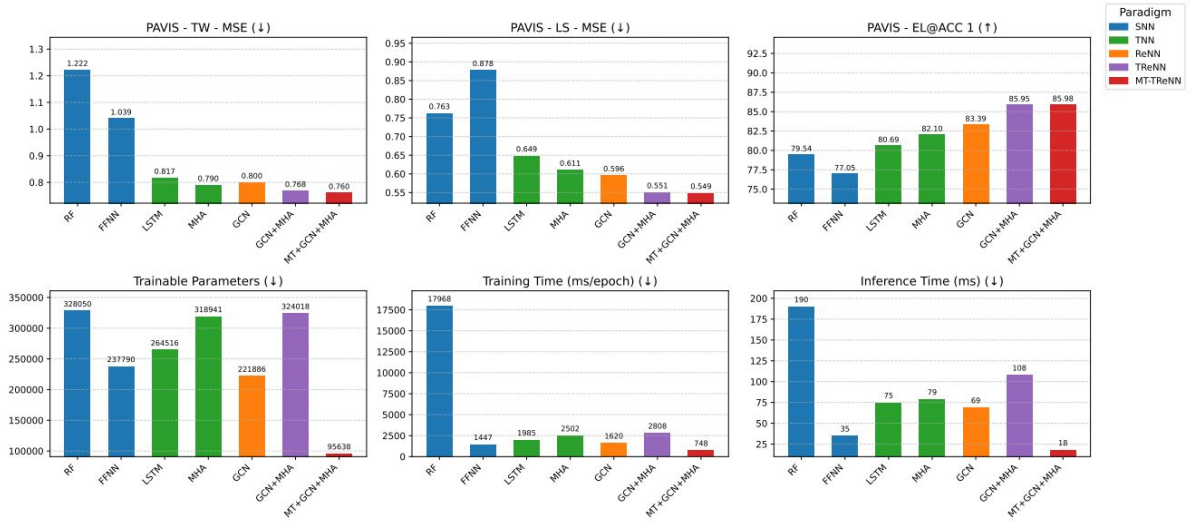


Fig. 10: The figure refers to the performance of the models under PAVIS. In the first row, they are respectively reported, MSE of TW, MSE of LS, and EL@ACC 1. In the second row on the other hand, they are reported trainable parameters required to achieve performances from the first row, the training time in ms required for a single training epoch, and the inference time required for a single team.

Preliminary results

Social Embedding and Multi-Task Learning

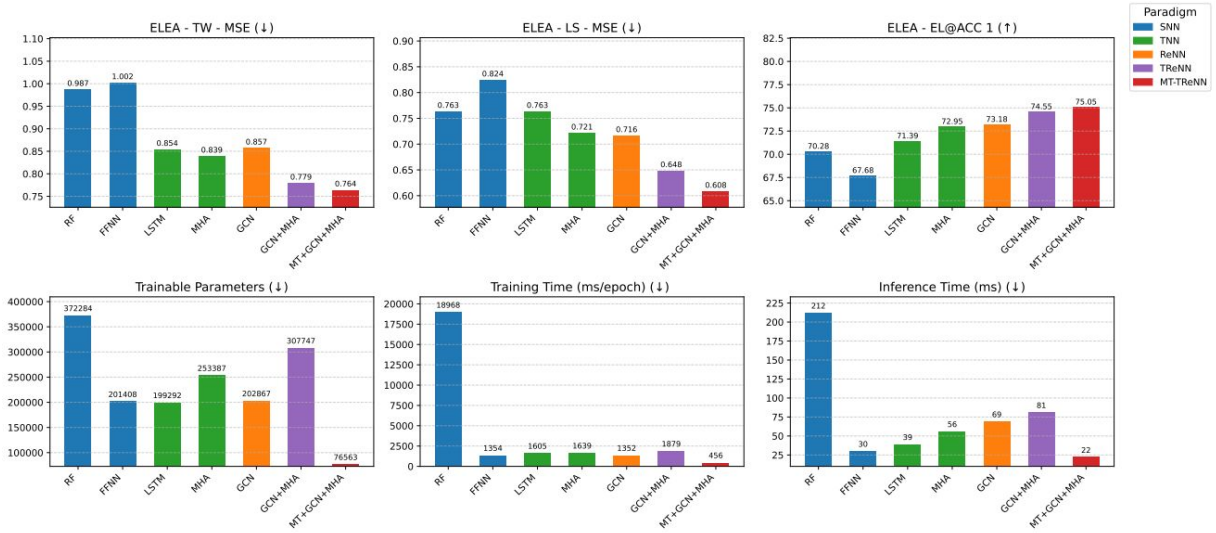


Fig. 11: The figure refers to the performance of the models under ELEA. In the first row, they are respectively reported, MSE of TW, MSE of LS, and EL@ACC 1. In the second row, on the other hand, they are reported as trainable parameters required to achieve performances from the first row, the training time in ms required for a single training epoch, and the inference time required for a single team.

Thank you for the attention